Monoid & Semigroup

Monoid · Fold invoice line totals / log strings · Build with Waqar

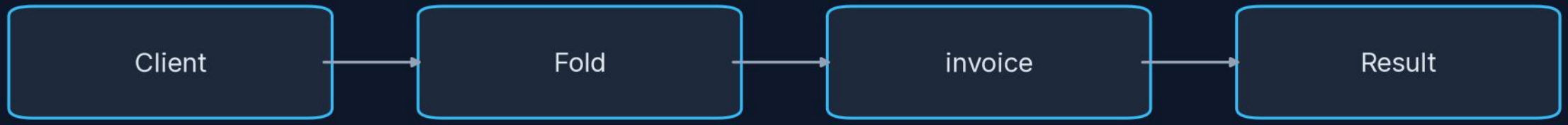
Today's Hook

- Problem: Fold invoice line totals / log strings
- Pattern: Monoid — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fold invoice line totals / log strings
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Monoid: Fold invoice line totals / log strings

Real-World Scenario

- Running example: Fold invoice line totals / log strings
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.monoid;

import com.buildwithwaqar.patterns.functional.monoid.Monoids;
import java.util.List;
/** Problem: fold invoice line totals and concatenate audit fragments. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Monoid problem: fold totals/logs ===");
    System.out.println("total=" + Monoids.INT_SUM.fold(List.of(100,250,50)));
    System.out.println("audit=" + Monoids.STRING.fold(List.of("reserve;", "charge;", "ship;")));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.monoid;

public final class Monoids {
    private Monoids() {}

    public static final Monoid<Integer> INT_SUM = new Monoid<>() {
        public Integer empty() { return 0; }
        public Integer combine(Integer a, Integer b) { return a + b; }
    };

    public static final Monoid<String> STRING = new Monoid<>() {
        public String empty() { return ""; }
        public String combine(String a, String b) { return a + b; }
    };
}
```

```java
package com.buildwithwaqar.patterns.functional.monoid;

import java.util.List;

interface Semigroup<A> {
    A combine(A a, A b);
}
```

Solution (2)

```
public interface Monoid<A> extends Semigroup<A> {
    A empty();

    default A fold(List<A> xs) {
        A acc = empty();
        for (A x : xs) acc = combine(acc, x);
        return acc;
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.monoid;

import com.buildwithwaqar.patterns.functional.monoid.Monoids;
import java.util.List;
/** Problem: fold invoice line totals and concatenate audit fragments. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Monoid problem: fold totals/logs ===");
    System.out.println("total=" + Monoids.INT_SUM.fold(List.of(100,250,50)));
    System.out.println("audit=" + Monoids.STRING.fold(List.of("reserve;", "charge;", "ship;")));
  }
}
```

Walkthrough Output

```
=== Monoid problem: fold totals/logs ===
total=400
audit=reserve;charge;ship;
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Monoid.

Related Patterns

- Contrast Monoid with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Monoid → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.